
AIA COPILOT TRAINING - CLAUDE CODE

Day 3: Orchestration & Production

Mastrorandi Produce | Wed, Aug 26,
2026

Scott Hay

Day 3: multiply it

01 MCP - connect to everything

9:00-10:30 - servers, scopes, auth, and when a script wins (Lab 3A)

02 Subagents & agent teams

10:40-12:10 - nesting, teams, and where they stop paying off (Lab 3B)

03 Production & CI/CD

12:55-2:20 - headless, the Action, hard cost controls (Lab 3C) - lunch 12:10

04 Scale, cost & new surfaces

2:30-3:40 - caching, model routing, usage attribution (Lab 3D)

05 Close

3:40-4:00 - the complete system and your week-one plan

WHERE YOU ARE

What you've built so far

Three days, one arc: from coding assistant to governed engineering system.



Day 1 - drive it - the loop, context, memory, permissions



Day 2 - teach it - Skills, hooks, invocation controls, validated plugin



Day 3 - multiply it - MCP, subagents, teams, headless CI



BLOCK 1 - 9:00-10:30

MCP: connect to everything

Servers, transports, auth, and the ToolSearch tradeoff.

BLOCK 1 - CONCEPT

What is MCP?

USB for AI - one protocol between any model and any tool.



Client - Claude Code requests capabilities



Server - exposes tools, resources, prompts



Protocol - one standard - write once, works everywhere

Three primitives, one server

Everything an MCP server offers is one of these.



Tools

The verbs - actions Claude can take.



Resources

The nouns - data Claude can read.



Prompts

The playbooks - guided workflows.



In practice most servers only ship Tools - Resources and Prompts are far less common in the wild.

MCP server - or just a script?



Plain script wins

- One-off, deterministic, single-step
- Bash runs it - zero protocol cost
- Reviewable like any repo code



MCP server wins

- Reused across sessions and teammates
- Auth, state, remote systems managed
- Claude discovers and calls it unprompted



Cheapest integration is a script in your repo - reach for MCP when reuse, auth, or remote reach demand it.

MCP tools load on demand by default

ToolSearch defers schemas until Claude needs a tool.



Default: deferred

Only tool names and server instructions start in context.



Loaded when relevant

ToolSearch discovers a tool; used schemas then enter context.



Know the exceptions

alwaysLoad, disabled ToolSearch, proxies, or unsupported providers load tools up front.



Use /context and /mcp to inspect the real session. Disable unused servers for security, reliability, and residual context cost.

stdio or Streamable HTTP?



stdio

- Local child process, same machine
- Full MCP functionality, no network
- Default - covers most Claude Code use



Streamable HTTP

- Remote/shared servers over HTTP
- SSE channel for server-to-client
- Sessions, auth, scaling to configure



SSE as a standalone transport is legacy - new remote servers should speak Streamable HTTP.

MCP Inspector: see it live



Launch

`npx @modelcontextprotocol/inspector` - opens a browser UI.



Connect

Point it at any MCP server, stdio or HTTP.



Test tools

Call tools by hand, see raw requests and responses.



Watch traffic

Inspect schemas, errors, and notifications as they flow.

MCP config: three scopes

Where the server config lives decides who gets it.



Personal

Your machine only - the default for `claude mcp add`.



Project-shared

`.mcp.json` in git - reviewable, versioned, team-wide.



Org-managed

IT-deployed policy decides what is allowed at all.



Project `.mcp.json` runs code on every teammate's machine - review it like a dependency change.

Paste this: wire up a server

What you are practicing

- Add a server with an exact command
- Scope it to the project, not global
- Authenticate from the shell - new

PASTE THIS

1

```
claude mcp add github -- npx -y  
@modelcontextprotocol/server-github
```

2

```
claude mcp add --transport http  
linear https://mcp.linear.app/mcp
```

3

```
claude mcp login linear
```

claude mcp login and logout

MCP auth is now a shell command, not a session ritual.



login

claude mcp login <server> -
OAuth from the terminal.



logout

claude mcp logout <server> -
revoke cleanly.



Why it matters

Scriptable auth for
onboarding and CI setup.



OAuth surface UNVERIFIED in July docs - re-verify claude mcp login flags at the delivery-morning gate.

Auth, OAuth & secret handling



OAuth remotes

Supported remote servers: claude mcp login runs the browser flow.



Not universal

Not every HTTP server speaks OAuth - some still want header tokens.



Env, not inline

stdio servers read tokens from env vars - values stay OUT of .mcp.json.



Never commit secrets

A token pasted into .mcp.json is a leak in git history forever.



OAuth flow specifics are UNVERIFIED in the July docs - re-verify claude mcp login at the morning-of gate.

Design tools for agents

An agent-facing tool is not a human-facing API.



Few, named well

Prefix by system: github_pr_*,
jira_issue_*



Corrective errors

Errors say 'try X instead', not
error code 4012.



Concise returns

Return what the task needs,
not everything.



Start with stdio locally, prove the tool works, then move transports - never debug both at once.

When NOT to add a tool



Shell already does it

Bash plus git covers it reliably - no tool needed.



Static knowledge

Facts belong in CLAUDE.md or a skill, not a tool call.



Workflow, not action

Multi-step procedure? That is a skill, not a tool.



Stop-and-ask

Wanting Claude to pause? That needs a dedicated tool, not a parameter.

MCP security in four lines



Least privilege

Read-only tokens unless write is truly needed.



Injection surface

Tool RESULTS enter context - a poisoned issue body is a prompt.



Trust like a dependency

An MCP server is code you run - vet it like an npm package.

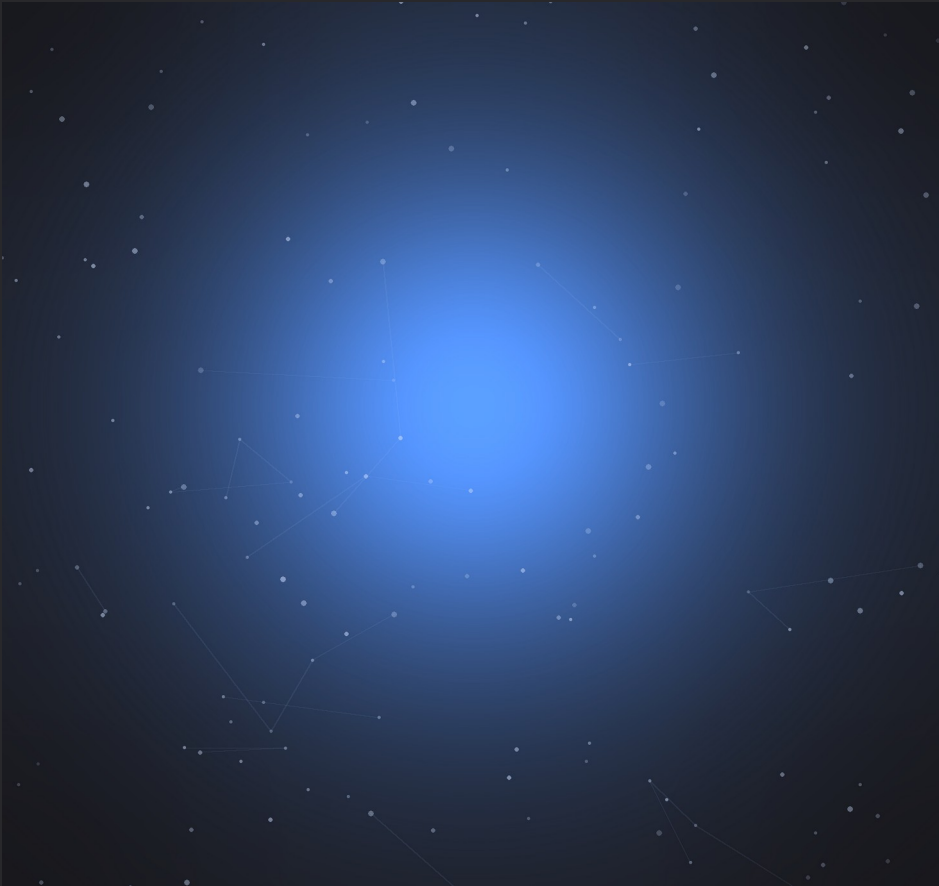


Scope the config

Project .mcp.json is reviewable in git - keep it there.

LIVE DEMO

Demo: MCP install to first call



WHAT WE'LL RUN TOGETHER

- 1** `claude mcp add + claude mcp list`
- 2** `claude mcp login`, then `/mcp` to verify
- 3** Ask Claude to use the new tool on a real task

LAB 3A - 25 MIN

Lab 3A: MCP end to end

Three lines to keep

If you remember nothing else about MCP, keep these.

“

ToolSearch defers MCP schemas by default - inspect exceptions with /context.

“

stdio unless the server lives somewhere else.

“

Tool results are model input - secure them like prompts.

BREAK

Break - 10 minutes



BLOCK 2 - 10:40-12:10

Subagents & agent teams

Nested delegation, background work, and teams - the day's centerpiece.

BLOCK 2 - CONCEPT

What is a subagent?

A focused worker with its own context - the point is compression, not just parallelism.



Isolated context - its exploration never pollutes yours



Own tools & model - scope it down, price it right



Reports back - a clean summary returns, not the noise

Delegate it or keep it?



Send to a subagent

- Bounded research or analysis
- Parallel independent checks
- Noisy exploration you only need summarized



Keep in main session

- Cross-cutting edits with shared state
- Work needing full conversation history
- Quick tasks - spawning costs overhead



Delegation is not free: each spawn reloads context - a 30-second task is cheaper done in place.

The built-in subagents



Explore

Fast read-only codebase search - grep that understands what it reads.



Plan

Read-only research for plan mode - architecture options, key files.



General-purpose

All tools, multi-step work - the capable, expensive one.



Your customs

Anything you define in `.claude/agents/` - next slide.

Custom subagents: 3 fields

A markdown file in `.claude/agents/` is a new team member.



Frontmatter

name + a pushy description of when to use it.



Tool allowlist

tools: Read, Grep - a reviewer that cannot write.



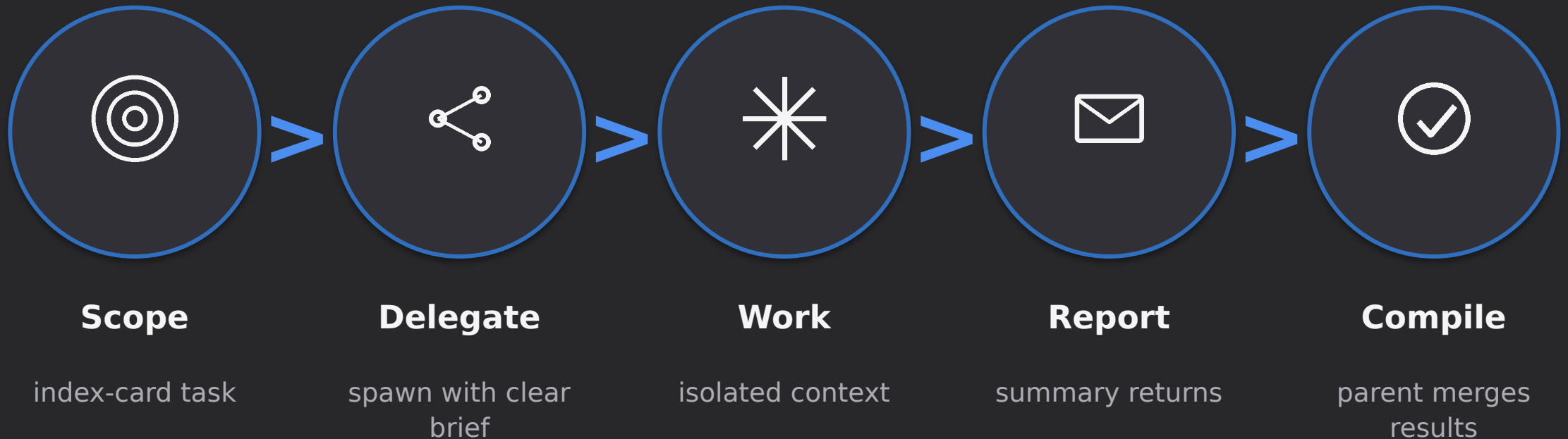
System prompt

The body is its standing instructions.



Pin a model per agent - Haiku graders and scanners cut cost 10x versus running everything on your main model.

The orchestration pattern



Nested subagents

Subagents can spawn subagents, so delegation becomes a bounded tree.



Agents all the way down

A coordinator can delegate detailed work and return one synthesis.



Three layers by default

CLAUDE_CODE_MAX_SUBAGENT_SPAWN_DEPTH changes the cap; 1 disables nesting.



Design shallow

Every layer adds a briefing, a report, latency, and spend.



Current default: up to three subagent layers below the main conversation; confirm the installed version before class.

Background subagents

The main agent keeps working while subagents run.



Non-blocking

Spawn the research, keep editing - no more waiting.



Subagent panel

Monitor the tree, open transcripts, and steer or stop a worker.



Collect later

Results integrate when ready, on your schedule.



As of v2.1.198, Claude runs subagents in the background by default unless a definition or task requires otherwise.

Fan-out with a grader



1. Fan out

Spawn N workers on the same brief - or N shards of it.



2. Work in parallel

Each drafts independently in its own context.



3. Grader scores

A dedicated subagent scores each result against a written rubric.



4. Keep the winner

Parent merges or picks - quality by tournament, not luck.



The 'Performance Outcomes' grader pattern is secondary-sourced - validate the workflow on your own repo before relying on it.

Worktree isolation

Parallel agents that WRITE need separate working copies.



One repo, N trees

git worktree gives each agent its own checkout.



No collisions

Writers cannot trample each other's edits.



Merge on your terms

Each tree is a branch - review, then merge.



Parallel writers need explicit isolation: set subagent ``isolation: worktree`` or partition files and use manual worktrees.

Paste this: nest and grade

What you are practicing

- Brief the lead, not the workers
- Name the depth you expect
- Point the grader at a rubric FILE

PASTE THIS

1

Launch a research lead subagent for the payments module. Have it spawn one subagent per service, then compile a single dependency report for me.

2

Spawn 3 subagents to each draft the retry logic independently. Then spawn a grader subagent that scores all 3 against rubric.md and reports the winner with scores.

Subagent anti-patterns



Inherited god-mode

Subagent gets the parent's full permissions - scope every allowlist.



No output contract

Undefined report format - the parent gets an essay it cannot merge.



Coupled subtasks

Workers that need each other's output should be ONE agent.



Five agents, tiny task

Orchestration overhead dwarfs the work - delegate when it pays.



Deep for depth's sake

Maxing the configured depth because you can - shallow trees usually win.

Case study: PR review pipeline



Security scanner

Read-only findings with file references and severity.



Logic reviewer

Correctness and edge cases against the PR intent.



Test-gap finder

Missing tests mapped to changed behavior.



Measure before scaling

Track review time, accepted findings, false positives, and cost per run.



The agent output starts human review; it does not approve or merge the PR.

BLOCK 2 - EXPERIMENTAL

Agent teams

Peers with a mailbox, not workers in a tree.



Lead agent - forms the team, assigns roles, merges



Teammates - parallel specialists sharing a task list and checkout



Mailbox - teammates message EACH OTHER directly

Enabling agent teams

Experimental flag on, then describe the team in plain English.



1. Enable

```
export  
CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=1
```



2. Describe

'Create a team: database, backend, frontend. Build the dashboard.'



3. Observe

Teammates claim tasks and message peers; partition files to avoid overwrites.



Experimental and disabled by default. Enable with `CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=1` and review current limitations before use.

Teams or subagents?



Agent team

- Multi-layer feature - DB + API + UI
- Interfaces must be NEGOTIATED between peers
- Parallel work only when file ownership is partitioned



Subagents

- Research, review, analysis - results roll UP
- Fan-out with a grader picking winners
- Stable today - no experimental flag



Default to subagents - reach for teams only when teammates genuinely need to talk to each other.

Where teams stop paying off

A team is the most expensive architecture in the room - know when it loses.



Cost

Every teammate is a full context and meter - N agents, N bills.



Latency

Mailbox coordination adds beats a single agent never pays.



Single-agent tasks

Sequential or same-file work gains nothing from peers.



If one agent does it in an hour, a team does it in ninety minutes - for several times the cost.

Choosing your architecture



Single session

95% of daily work - one domain, sequential, you stay in control.



+ Subagents

Bounded research, parallel checks, fan-out + grader - results roll up.



+ Agent team

Peers negotiating interfaces across layers - experimental, watch the cost.

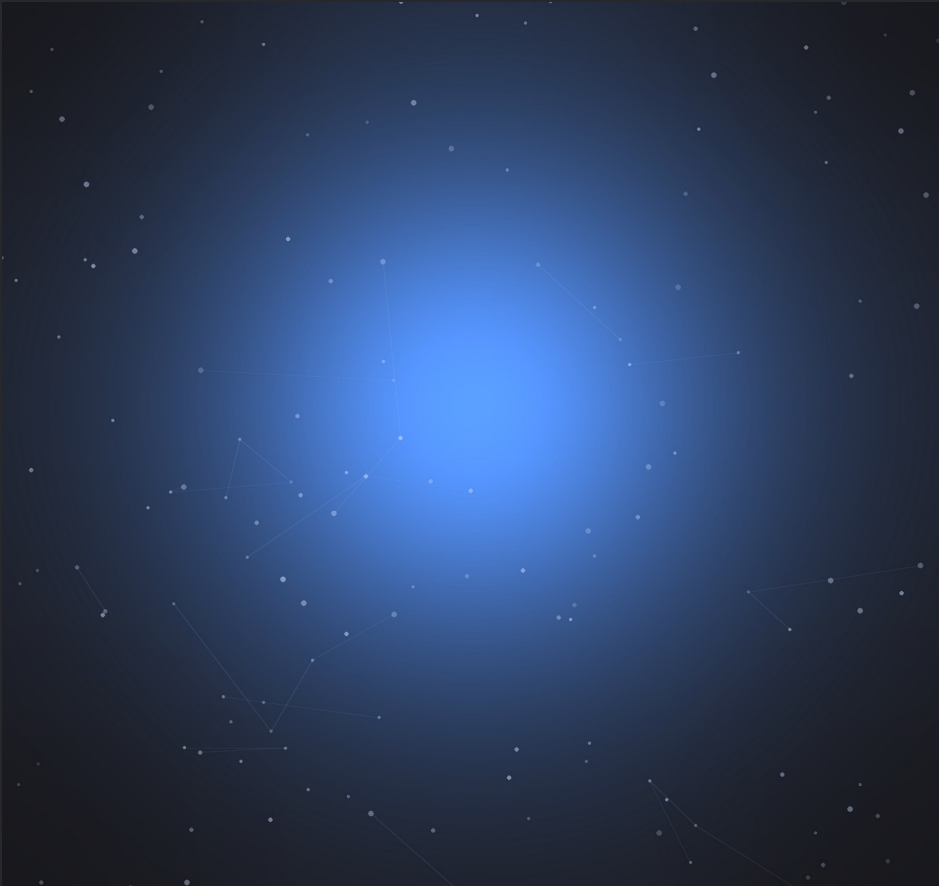


The gradient

Escalate only when the cheaper tier visibly fails the task.

LIVE DEMO

Demo: nested + background



WHAT WE'LL RUN TOGETHER

- 1** Nested delegation - watch the tree form
- 2** Background subagent runs while we continue in the main session
- 3** Fan-out x3 with a grader verdict

LAB 3B - 25 MIN

Lab 3B: subagents & teams

Three lines to keep

The centerpiece block in three sentences.

“

Delegation is compression
- buy summaries, not
noise.

“

Nest because the problem
decomposes, not because
you can.

“

Teams are for peers that
must talk - default to
subagents.

LUNCH

Lunch - 45 minutes



BLOCK 3 - 12:55-2:20

Production & CI/CD

Claude Code with nobody at the keyboard -
safely, on a budget.

BLOCK 3 - CONCEPT

Headless mode: `claude -p`

Print mode: prompt in, result out, exit - Claude Code as a Unix tool.



Non-interactive - runs in CI, cron, webhooks - no TTY



Structured output - JSON envelope: result, cost, duration



Bounded by flags - tools, turns, permissions - all explicit

Paste this: headless runs

What you are practicing

- Bound every run - turns and tools
- Ask for JSON when a script reads it
- Pipe diffs in, not whole repos

PASTE THIS

1

```
claude -p "Review src/ for error  
handling gaps" --max-turns 5 --  
allowedTools "Read,Grep,Glob"
```

2

```
claude -p "List all TODOs with file  
paths" --output-format json
```

3

```
git diff main...HEAD | claude -p "Review  
this diff. Flag critical issues only."
```

The four flags that matter



--output-format

text, json, or stream-json - scripts want the JSON envelope.



--max-turns

Hard cap on loop iterations - your runaway-cost brake.



--allowedTools

Explicit allowlist - 'Read,Grep,Glob' makes review runs read-only.



--permission-mode

plan for propose-only; never bypass permissions in shared CI.



Also pin --model in CI - an accidental Opus default turns a \$0.05 review into a \$0.50 one.

The official GitHub Action

anthropics/claude-code-action@v1 - PR review as a workflow step.



Trigger

on: pull_request - opened and synchronize.



Secret

ANTHROPIC_API_KEY from GitHub Secrets, never inline.



Output

Review lands as a PR comment your team reads.



Start the Action on ONE busy repo for two weeks - measure cost and signal before rolling it out org-wide.

Advisory or gating?



Advisory

- Posts a comment, never blocks merge
- Zero deployment risk if Claude is wrong
- The right STARTING point for every team



Gating

- Exit 1 on critical findings blocks merge
- Branch protection enforces the check
- Earn it: gate only rules you trust



The technical difference is one if-statement on the JSON output - the organizational difference is trust.

CI cost controls



Pin a mid-tier model

Automated review does not need your frontier model.



Feed diffs, not repos

git diff main...HEAD is the input - scope is your biggest lever.



Cap turns + output

--max-turns 5; cap output tokens - reviews are not essays.



Skip conditions

Lockfiles, generated code, docs-only PRs: skip the review entirely.



Log every run

total_cost_usd from the envelope into a dashboard - trend it.

Hard cost controls

01

Bound

--max-turns on
every run.

>

02

Fail safe

Limit hit, job
stops.

>

03

Alert

Spend threshold
pings a human.

>

04

Kill switch

One command
disables it.

Idempotency & retries

CI re-runs are a fact of life - make them safe and cheap.



Safe re-run

Check for a prior marker before acting - never duplicate work.



Retry transient only

Retry infra failures; never blind-retry model output - every retry bills.



Leave a marker

A comment tag or SHA file tells the next run: already reviewed.



Idempotency guards cost AND side effects - a re-run must not post twice, bill twice, or edit twice.

BLOCK 3 - COST

Three automation meters

Budget from the credential and runtime actually used.



Claude Code subscription identity -

Interactive and print-mode usage follows the live plan allowance; inspect /usage.



Eligible Agent SDK plan user - Separate monthly SDK credit may apply and refreshes with the billing cycle.



API or cloud identity - Token billed; provider budgets, telemetry, and hard caps are authoritative.

CI security posture



Secrets discipline

API key in GitHub Secrets, environment-scoped, rotated quarterly.



PRs are hostile input

A malicious diff is a prompt injection - never gate on untrusted forks.



No Bash in review

Read, Grep, Glob only - a reviewer must not execute the code it reads.



Least-privilege runner

Read-only checkout token; the Action needs no push rights.

If the pipeline is compromised

Size the blast radius before the incident, not after.



What leaks

The API key, repo contents,
every secret the job can read.



What executes

Whatever the allowlist
permits - why review jobs get
no Bash.



What contains it

Least-privilege tokens,
scoped secrets, read-only
checkout, rotation.



Assume the prompt is hostile and the key will leak - design so that day is boring, not existential.

Managed settings at scale



**managed-
settings.json**

IT-deployed, locked - developers cannot override it.



**NEW: managed-
settings.d/**

Multiple policy files, merged alphabetically - security team owns 10-security.json.



Hierarchy

Managed beats user (~/.claude) beats project (.claude/).



Deny wins

deny evaluates before ask and allow - everywhere, always.



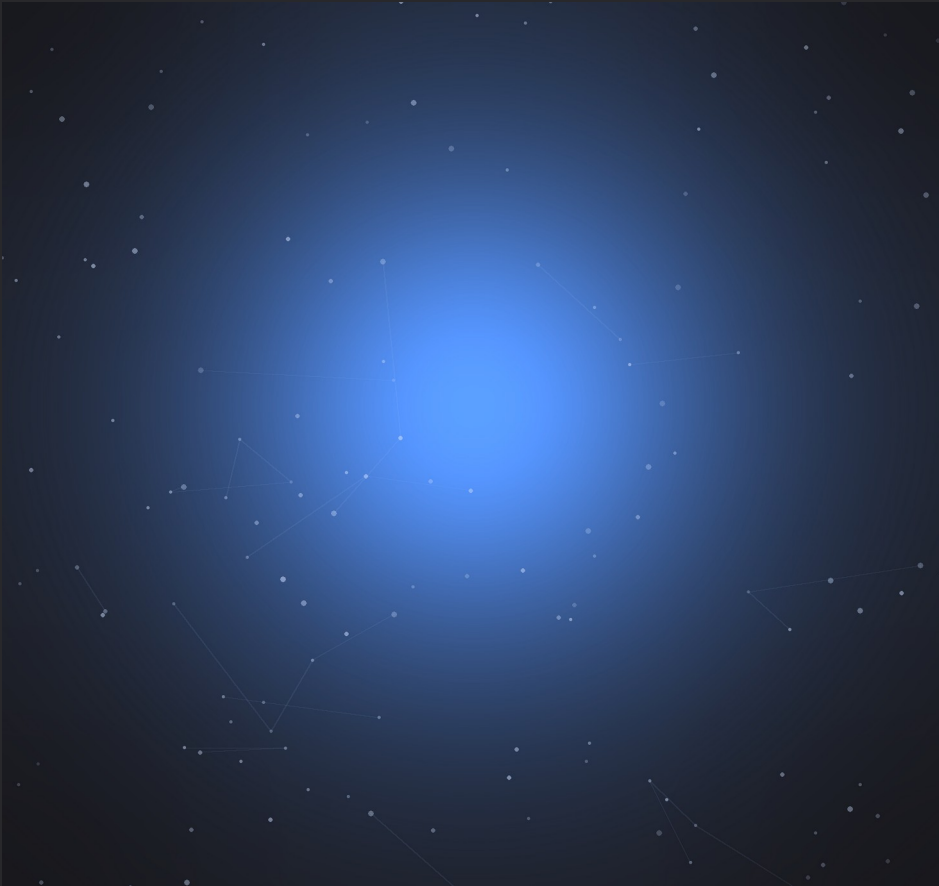
Alphabetical merge order is the contract - prefix files 10-, 20-, 30- so policy precedence is explicit.

Production permission profile

RULE	WHY IT EXISTS
deny Bash(rm -rf *), Bash(sudo *)	Destructive commands never run unattended
deny Read(.env), Read(.env.*)	Secrets stay out of model context entirely
deny Bash(curl * bash)	No piping the internet into a shell
allow Read, Grep, Glob	Review jobs are read-only by construction
ask Write, Edit	Mutations need a human or an explicit CI allowlist

LIVE DEMO

Demo: headless + the Action



WHAT WE'LL RUN TOGETHER

- 1 `claude -p review` with full guardrail flags
- 2 Parse the JSON envelope with `jq`
- 3 Open a PR - watch the Action review it

LAB 3C - 25 MIN

Lab 3C: your CI pipeline

Three lines to keep

Production in three sentences.

“

Headless, the flags ARE the permission system.

“

Start advisory - gating is earned, one rule at a time.

“

Name the credential, meter, budget owner, and stop condition for every automated run.

BREAK

Break - 10 minutes



BLOCK 4 - 2:30-3:40

Scale, cost & new surfaces

Caching, parallel worktrees, monitoring - and the June surprises.

BLOCK 4 - CONTEXT

The token problem

Fixed overhead and repeated context still multiply across agents, even with deferred MCP schemas.



Tool surface - Built-ins, server instructions, and ToolSearch metadata; schemas load when relevant or configured alwaysLoad.



Project context - CLAUDE.md, memory, files, and transcript consume the active window.



Every agent - Each subagent or teammate receives its own context and usage meter.

Cache-friendly prompt layout

01

System

Static, locked -
cached first.

>

02

Tools

Stable set and
order.

>

03

History

Grows, cheap
after prefix.

>

04

Input

Last - never
cached.

Cache-killers to avoid



Timestamps in system prompt

One changing character invalidates the whole prefix.



Model switch mid-session

Cache is model-specific - switching rebuilds it from zero.



Tool churn

Adding/removing servers or reordering tools breaks the prefix.



Editing CLAUDE.md mid-flight

Great between sessions, expensive during one.



Counter-intuitive: 100K tokens into an Opus session, 'saving money' by switching to Haiku is MORE expensive - full cache rebuild.

Compaction & context hygiene

The pressure valves for a filling context window.



/compact

Summarizes the transcript in place - cheap, thanks to the cached prefix.



/context

Shows where tokens sit - run it before long stretches.



Fresh session

New task, new session - the cheapest fix of all.



Sonnet 5's 1M window raises the ceiling - it does not repeal hygiene; big contexts still cost real money per turn.

Route models by task class

TASK CLASS	ROUTE TO
Mechanical - sweeps, graders	Cheapest tier (Haiku-class)
Daily implementation and debugging	Mid-tier default (Sonnet-class)
Spec, architecture, and critique passes	Premium (Opus-class) - worth it
Long autonomous runs	Highest tier your org approves

Paste this: parallel worktrees

What you are practicing

- One checkout per parallel session
- Independent branches, zero collisions
- Manual worktree isolation is explicit; subagents and teammates do not all receive it automatically

PASTE THIS

1 `git worktree add ../app-feature-a feature-a`

2 `git worktree add ../app-bugfix-b bugfix-b`

3 `cd ../app-feature-a && claude #
session 1 - repeat in b for session 2`

Usage data: attribute every dollar



Envelope metrics

total_cost_usd + duration_ms from every headless run into a dashboard.



/usage + /context

The in-session meters - plan allowance and window budget.



Tag every workflow

Name runs per pipeline so spend maps to a workflow, not a mystery.



Alert on 10x

Cost-per-run jumping an order of magnitude means something broke.

Artifacts: live interactive pages

Publish session output to a private page, then share deliberately.



Private by default

New artifacts start visible only to you on claude.ai.



Plan-aware sharing

Pro/Max can share publicly; Team/Enterprise can share inside the organization and may enable public links.



Connector-backed pages

MCP connectors run as each viewer; side effects use that viewer's account and permissions.



Available on Pro, Max, Team, and Enterprise. Publishing asks permission; connector-backed artifacts cannot be public.

Remote Control + Trusted Devices

Steer a local Claude Code session from claude.ai or the Claude apps while execution stays local.



Remote Control

Research preview on Pro, Max, Team, and Enterprise; API-key sessions and third-party cloud endpoints are unsupported.



Team/Enterprise control

Off by default until an Owner enables it; transcripts sync through Anthropic servers.



Trusted Devices beta

Team/Enterprise admin control requiring enrolled devices and a recent sign-in, with biometric/passkey step-up after 18 hours.



Filesystem and execution stay local; the synchronized transcript is stored under Anthropic data-usage policy.

New surfaces: honorable mentions



Built-in browser (wk 28)

Desktop app gains a browser - agents verify UI work visually.



Desktop on Linux (beta)

Ubuntu 22.04+/Debian 12+ since late June.



Cowork / Slack / Chrome

The wider Claude ecosystem - real, growing, out of scope today.



Out of scope by design - this course is the CLI-to-CI path; the ecosystem deserves its own session.

Where it still struggles



Very large changes

15+ files in one shot: errors compound - decompose instead.



Novel frameworks

Unfamiliar APIs invite hallucination - pin docs into context.



Ambiguous specs

Vague prompts produce confident, vague code.



Judgment calls

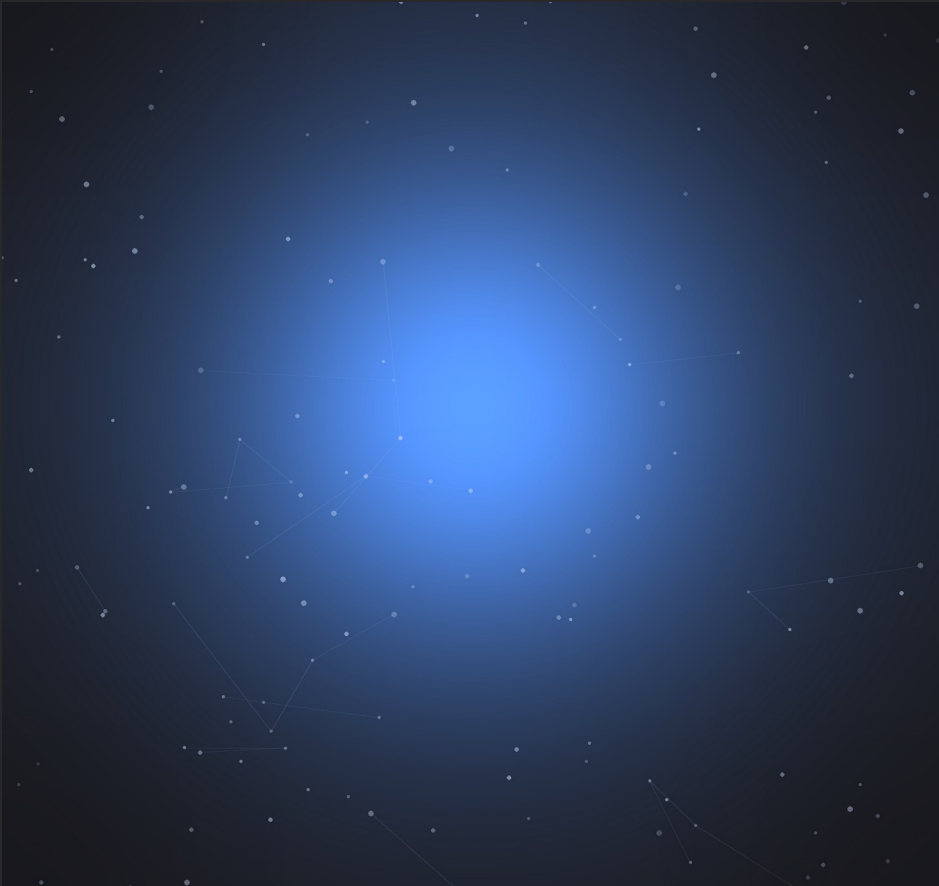
Architecture and tradeoffs still need your business context.

DISCUSSION - 10 MIN

Debate: one agent, many, or a team?

LIVE DEMO

Demo: worktrees + Artifacts



WHAT WE'LL RUN TOGETHER

- 1** Two worktrees, two live sessions
- 2** /context in both - overhead compared
- 3** Publish a session report as an Artifact

LAB 3D - 25 MIN

Lab 3D: capstone

Three lines to keep

Scale and cost in three sentences.

“

Caching is prefix-matching
- keep the static layers
still.

“

Pick the model per
session, never per mood.

“

New surfaces are plan-
dependent - verify before
you promise.



CLOSE - 3:40-4:00

Close: the complete system

Three days assembled, and your first week with it.

Three days, one system

Every layer you built, bottom to top.



Day 1 - Drive

The loop, CLAUDE.md, context, checkpoints, permissions.



Day 2 - Teach

Skills, hooks, invocation controls, validated plugin



Day 3 - Multiply

MCP, nested subagents, teams, headless CI, new surfaces.

Your week-one plan

01

Mon

CLAUDE.md
+ rules.

>

02

Tue

Wire one
server.

>

03

Wed

One
subagent.

>

04

Thu

One
headless
run.

>

05

Fri

Teach a
teammate.

The follow-on path



Anthropic Academy

Free certificated courses - Claude Code in Action, MCP Advanced.



Official docs + changelog

code.claude.com - re-check the changelog before big rollouts.



Community

Discord and the plugin marketplace - real workflows, weekly.



Agent SDK

When headless outgrows CI scripts - the programmatic path we referenced.

CLOSE

Three lines to leave with

The whole course, if you keep only this.

“

Drive one agent well
before you multiply it.

“

Your standards live in files
- skills, hooks, rubrics,
CLAUDE.md.

“

Escalate architecture only
when the cheaper tier
fails.

THANK YOU

Go multiply it

Your artifacts

Toolkit repo with CLAUDE.md, skill, hook, command, plugin, subagent, rubric, and CI workflow - plus lab guides, prompt packet, and the quick-reference handouts.

What happens next

Week-one plan starts Monday. Follow-up office hours are scheduled - bring your real-repo questions and one workflow you want reviewed.